Mohaeng(모행)

2025.12.30 - 2026.05.12 · 운영 종료

BE 1명, FE 1명, APP 1명, AI 2명, 디자인 1명

Node.js | Nest.js | TypeORM | BullMQ | PostgreSQL | MySQL | Redis | Docker | AWS | EC2 | S3

모행은 여행 계획 수립 시간을 줄이기 위한 AI 기반 여행 일정 추천 서비스입니다. 여행 스타일, 예산, 동행 유형, 선호 지역과 날씨·관심사 데이터를 반영해 맞춤형 일정을 추천합니다.

담당 역할

- **아이디어 제안과 팀원 모집부터 PM 역할, 백엔드 설계·구현·배포·운영까지 주도**
- 유일한 백엔드 개발자로서 API 설계, 인증, AI 비동기 처리, DB 이전, 배포 자동화와 장애 대응 흐름 책임

주요 구현

- ### 기능 개발부터 배포까지

  초기 MVP 단계에서 백엔드 전 과정을 1인이 담당하며, 기능 추가와 운영 준비가 동시에 필요.
  API 구현뿐 아니라 DB 변경, 배포, 장애 확인까지 반복 가능한 개발 흐름 구축 필요.
  Nest.js 기반 서버 구조, 핵심 API, DB 마이그레이션, Docker/Nginx/GitHub Actions 배포 흐름 구축.
  약 3개월간 핵심 기능 설계·구현·배포 완료, 2026 AI EXPO 출품 가능한 운영형 서비스로 완성.

- ### AI 비동기 처리 파이프라인 구축

  AI 일정 생성은 처리 시간이 길고, Node 서버와 Python 서버 사이의 네트워크·로직 실패 가능성 존재.
  사용자 요청을 막지 않으면서 실패·무응답 작업을 추적하고 복구할 구조 필요.
  BullMQ 큐, HTTP Callback, polling, retry/backoff, Stale Job 정리 기준을 분리해 적용.
  API 블로킹 제거, 작업 상태 추적, 실패 확정·재시도 흐름 확보로 AI 처리 안정성 개선.

- ### 배포 자동화·장애 대응 구조 구축

  AWS 배포 과정에서 compose 파일 누락, 이미지명 전달 오류, 컨테이너 내부 DB host 설정 오류 발생.
  수동 확인에 의존하면 배포 실패 원인 파악과 장애 감지가 늦어지는 문제 존재.
  Docker Compose, Nginx, GitHub Actions 배포 검증, DB host 설정, Discord 알림 흐름 정리.
  배포 실패 지점 추적과 장애 알림 흐름 표준화, 1인 운영 환경의 대응 속도 개선.

- ### Redis 기반 인증·보안 고도화

  JWT만으로는 토큰 탈취·재사용 상황에서 서버가 즉시 세션을 끊기 어려운 한계 존재.
  OTP, OAuth 일회용 코드, rate limit, refresh token 갱신을 중앙에서 제어할 저장소 필요.
  Redis에 인증 코드·요청 제한·refresh token 해시 저장, Rotation 시 기존 토큰 교체 구조 구현.
  해시 불일치 시 사용자 토큰 전체 폐기 가능, 재사용 공격 대응과 인증 흐름 안정성 개선.

- ### 피드백 수집·운영 알림 체계 구축

  초기 운영 단계에서 사용자 불편 사항이 흩어지면 개선 우선순위 판단이 어려운 상황.
  서비스 내부 피드백을 개발자가 즉시 확인하고 후속 작업으로 연결할 구조 필요.
  피드백 수집 API와 Discord Webhook 연동, 운영 채널 실시간 전달 흐름 구현.
  사용자 의견 확인 속도 개선, 운영 피드백을 서비스 개선 작업으로 연결하는 루프 완성.

성능 개선 및 트러블 슈팅

- **좋아요 조회 N+1 제거**

  코스·블로그 목록 조회 후 각 항목의 isLiked를 개별 조회해 페이지 크기만큼 DB round-trip 증가. 항목 ID를 모아 userId + itemIds 기준으로 일괄 조회하고 Set으로 매핑해 **항목 10개 기준 11회, 20개 기준 21회였던 쿼리 수를 2회로 감소**.

- **AWS 이전 중 레거시 DB 이슈 해결**

  AWS 이전 과정에서 레거시 FK 제약과 MySQL·PostgreSQL 간 Date 처리 차이로 배포 환경 오류 발생. 문제 FK를 정리하고 Date 저장·비교 기준을 표준화해 로컬·운영 DB 간 동작 차이와 마이그레이션 리스크 감소.

주요 성과

- **백엔드 1인 개발로 핵심 기능 설계·구현·배포 완료**
- **기능 우선순위와 일정 흐름을 조율해 6인 팀의 MVP 개발 완료에 기여**
- **2026 AI EXPO 출품**

문제 발견과 의사결정

- AI 일정 생성은 처리 시간이 길지만 초기 트래픽 규모에서는 SSE나 WebSocket의 연결 유지·유실 대응 비용이 더 크다고 판단. 우선 polling으로 구현 복잡도를 낮추고, 작업 상태는 서버에서 관리해 추후 실시간 알림 구조로 확장 가능하게 설계.

- JWT만으로는 refresh token 탈취·재사용 상황에서 서버가 즉시 세션을 무효화하기 어렵다고 판단. refresh token 원문 대신 해시를 Redis에 저장하고 Rotation 구조를 적용해 재사용 탐지와 강제 폐기 가능성 확보.

- Google Place API 정책상 장소 데이터 무기한 저장 불가 문제 확인. 매 요청마다 API를 호출하면 비용과 응답 시간이 커지므로, 장소 updated_at 기준 30일 캐시 갱신 구조로 정책 준수와 호출 비용 최적화를 함께 해결.

- 좋아요 여부 조회는 Promise.all로 병렬화해도 DB round-trip 수가 페이지 크기만큼 증가하는 구조는 그대로라고 판단. 응답 속도보다 쿼리 수 자체를 줄이는 것이 핵심이라 보고, batch query + Set 매핑 방식으로 전환.

회고 및 느낀 점

- 모행은 직접 떠올린 아이디어에서 시작해 팀원을 모집하고, 일정 조율과 기능 방향을 정리하며 PM 역할까지 함께 맡아 주도적으로 이끌어간 프로젝트입니다. 서버 1인 개발자로 인증, AI 비동기 처리, DB, 배포, 장애 대응까지 책임져야 했기 때문에 많은 시간과 노력이 필요했습니다. 이 과정에서 AI 도구를 적극 활용해 생산성을 높였지만, 최종 설계와 검증은 직접 책임지며 운영 가능한 구조를 만드는 경험을 쌓았습니다.